seamlessly with MongoDB. You can install it via Composer by running the command:

```
composer require mongodb/laravel-mongodb
```

Once installed, you can begin configuring your Laravel application to use MongoDB as its database backend.

12.4 Configuring MongoDB in Laravel

After installing the MongoDB package, the next step is to configure the Laravel project to use MongoDB. This is done by updating the `.env` file to set the MongoDB connection parameters such as host, port, and database name. For example:

```
DB_CONNECTION=mongodb
MONGODB_URI=
MONGODB_DATABASE=
```

You must also register the MongoDB connection inside the `config/database.php` file. Add a new connection array with the driver set to `mongodb` and map the environment variables accordingly. With this setup, Laravel is now ready to use MongoDB as its primary or secondary database connection.

12.5 Creating a MongoDB Model

To work with MongoDB collections in Laravel, you need to define models that extend the MongoDB-compatible Eloquent model provided by the package. Instead of using the base `Illuminate\Database\Eloquent\Model`, your model should extend `MongoDB\Laravel\Eloquent\Model`. For instance, a `Product` model might look like this:

```
use MongoDB\Laravel\Eloquent\Model;

class Product extends Model
{
    protected $connection = 'mongodb';
    protected $collection = 'products';
    protected $fillable = ['name', 'price', 'category'];
}
```

This configuration tells Laravel to use the MongoDB connection and map the model to the `products` collection. The `$fillable` array defines which fields can be mass-assigned, similar to standard Eloquent behavior.